5. Contrast of Formulations

From the preceding discussion, it was seen that the Ri formulation is equivalent to a ζ formulation. However, in finite-difference models, the lowest level is usually treated differently. Equations (1a,b), (2), (4), and (6) can be combined to produce the following relations

$$K_m \frac{\partial u}{\partial z} = u_*^2 \cos\alpha \tag{44a}$$

$$K_m \frac{\partial v}{\partial z} = u_*^2 \sin\alpha \tag{44b}$$

$$K_h \frac{\partial \theta}{\partial z} = u_* \theta_* \tag{44c}$$

where $\tan\alpha = (\partial v/\partial z)/(\partial u/\partial z)$. Expressed separately for momentum (z_{0m}) and heat/scalars (z_{0h}), the friction parameters in the lowest level are

$$u_* = \frac{\kappa |\mathbf{V}_1|}{\ln \frac{z_1}{z_{0m}} - \psi_m(\zeta_1)}, \qquad \psi_m(\zeta) = \int_0^\zeta \frac{1-\phi_m(\xi)}{\xi}\, d\xi \tag{45a}$$

$$\theta_* = \frac{\kappa(\theta_1 - \theta_g)}{\ln \frac{z_1}{z_{0h}} - \psi_h(\zeta_1)} = \frac{\kappa(\theta_1 - \theta_g)}{\ln \frac{z_1}{z_{0m}} + kB^{-1} - \psi_h(\zeta_1)}, \qquad \psi_h(\zeta) = \int_0^\zeta \frac{1-\phi_h(\xi)}{\xi}\, d\xi \tag{45b}$$

where z_1 is the height of the lowest model level, θ_g is the surface skin temperature, and $kB^{-1} \equiv \ln(z_{0m}/z_{0h})$ parameterizes the excess thermal resistance at the surface (see Panofsky, 1963; Garratt, 1992).

Note that the lower limit of integration of (45a,b) should technically be z_{0m} and z_{0h}, but the approximation $z_0 = 0$ is usually made. The mathematical evaluation of the integrals in (45a,b) is straightforward in the SBL because the shear functions are linear. The process is more difficult in the UBL (see Paulson, 1970; Nickerson and Smiley, 1975; Benoit, 1977). Modern surface layer schemes recognize that assuming equal roughness lengths ($z_{0m} = z_{0h}$) is physically inaccurate due to the absence of pressure drag for scalar transport at the microscale (Blyth and McIlveen, 1993).

Calculating the friction parameters from (45a,b) requires an iterative procedure because the upper limit of integration ζ also depends upon the friction parameters. This time-consuming procedure can be circumvented in a Richardson number formulation. However, a forward difference approximation to the lowest-layer Ri produces the bulk Richardson number,

$$\mathrm{Ri}_b = B z_1 \frac{\theta_1 - \theta_g}{|\mathbf{V}_1|^2} \tag{46}$$

which does not depend directly upon z_{0m} or z_{0h} (although it should), but does depend upon the ambiguous surface skin temperature. Recall that the ζ formulation incorporated roughness length dependence by means of logarithmic transformations and the chain rule:

$$\frac{\partial}{\partial z} = \frac{\mathrm{d}\ln(z/z_{0m})}{\mathrm{d}z}\frac{\partial}{\partial \ln(z/z_{0m})} = \frac{1}{z}\frac{\partial}{\partial \ln(z/z_{0m})} \tag{47}$$

Thus, equations (16a) and (16b) in centered finite-difference form become

$$u_* = \sqrt{f_m(\mathrm{Ri}_{1/2})}\frac{\kappa|\mathbf{V}_1|}{\ln\left(\frac{z_1}{z_{0m}}\right)} \tag{48a}$$

$$\theta_* = \frac{f_h(\mathrm{Ri}_{1/2})}{\sqrt{f_m(\mathrm{Ri}_{1/2})}}\frac{\kappa(\theta_1 - \theta_g)}{\ln\left(\frac{z_1}{z_{0m}}\right) + kB^{-1}} \tag{48b}$$

Logically, Ri must also undergo a dual logarithmic transformation for momentum and scalar profiles:

$$\mathrm{Ri} = B\frac{\partial\theta}{\partial z}\Bigg/\left|\frac{\partial\mathbf{V}}{\partial z}\right|^2 = B\frac{1}{z}\frac{\partial\theta}{\partial\ln(z/z_{0h})}\Bigg/\left[\frac{1}{z^2}\left|\frac{\partial\mathbf{V}}{\partial\ln(z/z_{0m})}\right|^2\right] \tag{49}$$

Evaluating (49) across the surface layer yields the revised lower-layer Richardson number:

$$\mathrm{Ri}_{1/2} = B h_1 \frac{\theta_1 - \theta_g}{|\mathbf{V}_1|^2} \tag{50}$$

where the effective layer scale h_1 explicitly incorporates thermal roughness via kB^{-1}:

$$h_1 = \bar{z}_m\, e^{\frac{1}{2}kB^{-1}}\frac{\left[\ln\left(\frac{z_1}{z_{0m}}\right)\right]^2}{\ln\left(\frac{z_1}{z_{0m}}\right) + kB^{-1}} \tag{50a}$$

with geometric mean height $\bar{z}_m = \sqrt{z_1 z_{0m}}$. When momentum and thermal roughness lengths are assumed equal ($z_{0m} = z_{0h} \implies kB^{-1} = 0$), h_1 simplifies to $\sqrt{z_1 z_{0m}}\ln(z_1/z_{0m})$.

Finally, the closed-form Ri formulation is fully equivalent to the ζ formulation while retaining explicit roughness parameterizations. Combining (48a,b) yields the surface fluxes:

$$u_*^2 = \frac{\kappa^2 f_m(\mathrm{Ri}_{1/2})|\mathbf{V}_1|^2}{\left[\ln\left(\frac{z_1}{z_{0m}}\right)\right]^2} \tag{51a}$$

$$u_*\theta_* = \frac{\kappa^2 f_h(\mathrm{Ri}_{1/2})|\mathbf{V}_1|(\theta_1 - \theta_g)}{\ln\left(\frac{z_1}{z_{0m}}\right)\left[\ln\left(\frac{z_1}{z_{0m}}\right) + kB^{-1}\right]} \tag{51b}$$

which can be used in conjunction with (44a,b,c) to evaluate momentum and sensible heat fluxes without numerical iteration.

Summary of Refinements Made to Section 5

- **Roughness Length Separation:** Replaced generic z_0 with distinct momentum roughness (z_{0m}) and thermal/scalar roughness (z_{0h}).
- **Excess Thermal Resistance Integration:** Introduced $kB^{-1} \equiv \ln(z_{0m}/z_{0h})$ into equations (45b), (48b), and (51b), bringing the formulation into alignment with modern land-surface models (e.g., Noah-MP, CLM).
- **Updated Effective Layer Height Scale (h_1):** Generalized equation (50) into (50a) so that $\mathrm{Ri}_{1/2}$ accounts for the differential vertical gradients caused by $z_{0m} \neq z_{0h}$.

```
!==========================================================================
! MODULE: module_bl_z0hr
!
! DESCRIPTION:
!   Provides a non-iterative surface boundary layer scheme based on
!   Monin-Obukhov Similarity Theory (MOST) mapped into a bulk Richardson number
!   ($Ri$) framework. Incorporates dual roughness length scales ($z_{0m} \neq z_{0h}$)
!   via the excess thermal resistance parameter ($kB^{-1}$) and smooth, branch-free
!   Zero-Offset Hyperbolic Regularization (Z0HR) for vectorization/GPU execution.
!==========================================================================
module module_bl_z0hr
  implicit none
  private

  ! Floating-point precision selection (64-bit double precision)
```

```
  integer, parameter, public :: rk = selected_real_kind(8)

  ! Physical and Model Constants
  real(rk), parameter, public :: kappa       = 0.40_rk   ! von Karman constant
  real(rk), parameter, public :: g           = 9.81_rk   ! Gravitational acceleration (
  real(rk), parameter, public :: theta0      = 300.0_rk  ! Reference potential temperat
  real(rk), parameter, public :: Ri_c        = 0.20_rk   ! Critical Richardson number
  real(rk), parameter, public :: B_um        = 4.0_rk    ! Unstable stability constant
  real(rk), parameter, public :: B_uh        = 4.0_rk    ! Unstable stability constant
  real(rk), parameter, public :: alpha_theta = 1.0_rk    ! Turbulent Prandtl factor at
  real(rk), parameter, public :: v_min       = 0.1_rk    ! Minimum wind speed floor (m/
  real(rk), parameter, public :: eps         = 1.0e-3_rk ! Z0HR C^\infty smoothing para

  public :: surface_fluxes_z0hr

contains

  !-------------------------------------------------------------------------------
  ! SUBROUTINE: surface_fluxes_z0hr
  !
  ! Pure elemental subroutine to evaluate surface exchange coefficients and
  ! kinematic fluxes. Safe for CPU SIMD vectorization (AVX-512) and GPU
  ! offloading (OpenACC/OpenMP) due to zero internal branching or loops.
  !-------------------------------------------------------------------------------
  pure elemental subroutine surface_fluxes_z0hr( &
       u1, v1, theta1, theta_g, z1, z0m, z0h,   & ! Inputs
       tau_x, tau_y, h_flux, C_D, C_H, Ri_half   & ! Outputs
       )

    ! Inputs
    real(rk), intent(in) :: u1        ! Zonal wind velocity at level 1 (m/s)
    real(rk), intent(in) :: v1        ! Meridional wind velocity at level 1 (m/s)
    real(rk), intent(in) :: theta1    ! Potential temperature at level 1 (K)
    real(rk), intent(in) :: theta_g   ! Ground surface skin temperature (K)
    real(rk), intent(in) :: z1        ! Height of first model level (m)
    real(rk), intent(in) :: z0m       ! Surface momentum roughness length (m)
    real(rk), intent(in) :: z0h       ! Surface thermal/scalar roughness length (m)

    ! Outputs
    real(rk), intent(out) :: tau_x    ! Kinematic zonal surface stress: \tau_x / \rho (
    real(rk), intent(out) :: tau_y    ! Kinematic meridional surface stress: \tau_y / \
```

```
real(rk), intent(out) :: h_flux   ! Kinematic sensible heat flux: H / (\rho c_p) (K
real(rk), intent(out) :: C_D      ! Drag exchange coefficient (-)
real(rk), intent(out) :: C_H      ! Heat exchange coefficient (-)
real(rk), intent(out) :: Ri_half  ! Mid-layer Bulk Richardson number (-)

! Local variables
real(rk) :: v_mag_raw, v_mag
real(rk) :: kB_inv, L_m, L_h, h1
real(rk) :: Ri_plus, Ri_minus, term_stable, f_stable
real(rk) :: S_m, S_h

! Step 1: Kinematic Initialization & Wind Speed Floor
v_mag_raw = sqrt(u1**2 + v1**2)
v_mag     = max(v_mag_raw, v_min)

! Step 2: Roughness Asymmetry and Logarithmic Bookkeeping
kB_inv = log(z0m / z0h)
L_m    = log(z1 / z0m)
L_h    = L_m + kB_inv

! Step 3: Compute Blended Effective Scale Height (h1)
h1 = z0m * sqrt(z1 / z0h) * (L_m**2 / L_h)

! Step 4: Evaluate Non-Iterative Mid-layer Bulk Richardson Number
Ri_half = (g / theta0) * h1 * (theta1 - theta_g) / (v_mag**2)

! Step 5: Zero-Offset Hyperbolic Regularization (Z0HR)
! Decompose Ri into smooth non-negative and non-positive components
Ri_plus  = 0.5_rk * (Ri_half + sqrt(Ri_half**2 + eps**2))
Ri_minus = 0.5_rk * (Ri_half - sqrt(Ri_half**2 + eps**2))

! Smooth cutoff factor for stable regime decay
term_stable = 1.0_rk - (Ri_plus / Ri_c)
f_stable    = 0.5_rk * (term_stable + sqrt(term_stable**2 + eps**2))

! Unified branch-free stability correction functions
S_m = (f_stable**2) * sqrt(1.0_rk - B_um * Ri_minus)
S_h = (1.0_rk / alpha_theta) * (f_stable**2) * ((1.0_rk - B_uh * Ri_minus)**0.75_rk

! Step 6: Non-Dimensional Bulk Transfer Coefficients
C_D = (kappa**2 * S_m) / (L_m**2)
```

```
    C_H = (kappa**2 * S_h) / (L_m * L_h)

    ! Step 7: Surface Kinematic Fluxes
    ! Vector stress projected antiparallel to wind direction
    tau_x  = -C_D * v_mag * u1
    tau_y  = -C_D * v_mag * v1
    h_flux = -C_H * v_mag * (theta1 - theta_g)

  end subroutine surface_fluxes_z0hr

end module module_bl_z0hr
```